

ใบงานที่ 7 โครงข่ายประสาทเทียมและการรู้จำใบหน้า (Neural Networks and Face Recognition)

วัตถุประสงค์การเรียนรู้

- เข้าใจกระบวนการทำ Data Preparation สำหรับข้อมูลประเภทรูปภาพและวิดีโอ
- เขียนโค้ดใช้งานโมเดล MTCNN เพื่อตรวจจับและตัดภาพใบหน้าได้
- เขียนโค้ดใช้งานสถาปัตยกรรม InceptionResnetV1 เพื่อสร้าง Face Embedding ได้
- ปรับปรุงค่าพารามิเตอร์ (Hyperparameters) เพื่อวิเคราะห์ผลลัพธ์ที่เปลี่ยนไปได้

เครื่องมือและอุปกรณ์

- เครื่องคอมพิวเตอร์ที่ติดตั้งโปรแกรม PyCharm IDE
- กล้อง Webcam สำหรับทดสอบระบบแบบ Real-time
- ไฟล์รูปภาพหรือวิดีโอตัวอย่างสำหรับการตรวจจับใบหน้า
- เอกสารใบความรู้หน่วยที่ 7
- แบบฟอร์มบันทึกผลการปฏิบัติงาน (ท้ายใบงาน)

ลำดับขั้นตอนการปฏิบัติงาน

คำชี้แจง: ให้ผู้เรียนสร้างไฟล์และเขียนโค้ดตามขั้นตอนต่อไปนี้ เพื่อสร้างระบบรู้จำใบหน้า (Face Recognition)

ขั้นตอนที่ 1: การเตรียมและปรับสเกลข้อมูล (Data Preparation)

ให้สร้างไฟล์ชื่อ ตัดภาพจากวิดีโอ.py และพิมพ์โค้ดเพื่อดึงภาพใบหน้าจากวิดีโอ

```
import cv2
import os
from facenet_pytorch import MTCNN
from PIL import Image
```

```
VIDEO_PATH = "videos/name.mp4"
SAVE_DIR = "ref_faces/name"
os.makedirs(SAVE_DIR, exist_ok=True)

mtcnn = MTCNN(image_size=160, margin=20)

cap = cv2.VideoCapture(VIDEO_PATH)
frame_count = 0
save_count = 0

while True:
    ret, frame = cap.read()
    if not ret:
        break
    frame_count += 1

    # เอาเฉพาะทุก ๆ 5 เฟรม
    if frame_count % 5 != 0:
        continue

    # ตรวจจับใบหน้า
    boxes, _ = mtcnn.detect(frame)
    if boxes is not None:
        for i, box in enumerate(boxes):
            x1, y1, x2, y2 = [int(v) for v in box]
            face_img = frame[y1:y2, x1:x2]
            pil_img = Image.fromarray(cv2.cvtColor(face_img,
cv2.COLOR_BGR2RGB)).convert('RGB')

            save_path = os.path.join(SAVE_DIR, f"frame_{frame_count}_{i}.jpg")
            pil_img.save(save_path)
            save_count += 1
```

```
cap.release()
print(f"บันทึกใบหน้าทั้งหมด {save_count} ภาพ")
```

ขั้นตอนที่ 2: การสร้างโมเดล Face Embedding

ให้สร้างไฟล์ชื่อ train.py เพื่อนำภาพใบหน้าที่ได้มาแปลงเป็นรหัสตัวเลขทางคณิตศาสตร์

```
import os, pickle, numpy as np
from PIL import Image
from facenet_pytorch import MTCNN, InceptionResnetV1

# โหลดโมเดลตรวจจับ + สร้าง embedding
mtcnn = MTCNN(image_size=160, margin=20)
resnet = InceptionResnetV1(pretrained='vggface2').eval()

refs = {}
for person in os.listdir("ref_faces"):
    person_dir = os.path.join("ref_faces", person)
    if not os.path.isdir(person_dir):
        continue

    embeddings = []
    for file in os.listdir(person_dir):
        if file.endswith((".jpg", ".png")):
            img = Image.open(os.path.join(person_dir, file)).convert("RGB")
            face = mtcnn(img)
            if face is not None:
                emb = resnet(face.unsqueeze(0)).detach().numpy()
                embeddings.append(emb)

    if embeddings:
        refs[person] = np.mean(embeddings, axis=0)

# เซฟโมเดล
```

```
with open("face_model.pkl", "wb") as f:
    pickle.dump(refs, f)
print("บันทึกโมเดลเสร็จสิ้น!")
```

ขั้นตอนที่ 3: การนำไปใช้งานจริง (Real-Time Inference)

ให้สร้างไฟล์ชื่อ run_model.py เพื่อเปิดกล้องและให้ AI ทายชื่อบุคคลแบบ Real-Time

```
import cv2, pickle, numpy as np
from PIL import Image
from facenet_pytorch import MTCNN, InceptionResnetV1

# โหลดตัวตรวจจับและโมเดล face embedding
mtcnn = MTCNN(image_size=160, margin=20)
resnet = InceptionResnetV1(pretrained="vggface2").eval()

# โหลด reference
with open("face_model.pkl", "rb") as f:
    refs = pickle.load(f)

def cosine_similarity(a, b):
    return np.dot(a, b.T) / (np.linalg.norm(a) * np.linalg.norm(b) + 1e-10)

cap = cv2.VideoCapture(0)
while True:
    ret, frame = cap.read()
    if not ret:
        break

    img = Image.fromarray(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
    boxes, _ = mtcnn.detect(img)

    if boxes is not None:
        for box in boxes:
```

```

x1, y1, x2, y2 = map(int, box)
face = img.crop((x1, y1, x2, y2))
face_tensor = mtcnn(face)
if face_tensor is None:
    continue

emb = resnet(face_tensor.unsqueeze(0)).detach().numpy()
# หาคนที่คล้ายที่สุด
best_name = "Unknown"
best_score = 0.0
for n, ref in refs.items():
    sim = cosine_similarity(emb, ref)
    if sim > best_score:
        best_name = n
        best_score = sim

if best_score > 0.6:
    final_name = best_name
    color = (0, 255, 0) # รู้จัก
else:
    final_name = "Unknown"
    color = (0, 0, 255) # ไม่รู้จัก

# วาดกรอบและชื่อ
cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)

# แสดงชื่อพร้อมคะแนนความมั่นใจ
score = float(best_score) * 100.0
text_display = f"{final_name} ({score:.2f}%)"
cv2.putText(frame, text_display, (x1, y1 - 10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.6, color, 2)

cv2.imshow("Face Recognition", frame)

```

```
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
```

แบบบันทึกผลการปฏิบัติงาน

ส่วนที่ 1: ผลลัพธ์การทำงานพื้นฐาน

คำชี้แจง: บันทึกผลลัพธ์ที่ได้จากการรันโค้ดตามขั้นตอนข้างต้น

1. ในขั้นตอนที่ 1 หลังจากรันไฟล์ ตัดภาพจากวิดีโอ.py โปรแกรมแจ้งว่า "บันทึกใบหน้าทั้งหมด ภาพ"
2. ในขั้นตอนที่ 3 เมื่อใบหน้าของคุณปรากฏบนกล่อง AI สามารถระบุชื่อของคุณได้ถูกต้องหรือไม่? (ถูกต้อง / ไม่ถูกต้อง)
3. หากมีบุคคลอื่นที่ไม่ได้อยู่ในชุดข้อมูลฝึกสอน (ref_faces) เข้ามาในกล่อง AI จะแสดงชื่อว่าอะไร?
.....

ส่วนที่ 2: การทดลองเชิงประยุกต์ (Challenge)

คำชี้แจง: ให้ผู้เรียนลองแก้ไขโค้ดบางจุด เพื่อสังเกตพฤติกรรมของ AI ที่เปลี่ยนไป แล้วบันทึกผลการทดลอง

1. การปรับความถี่ในการดึงภาพ (Frame Sampling):
 - กลับไปที่ไฟล์ ตัดภาพจากวิดีโอ.py ให้ลองเปลี่ยนเงื่อนไข if frame_count % 5 != 0: เป็น if frame_count % 30 != 0: แล้วรันโค้ดใหม่
 - จำนวนภาพที่บันทึกได้ เปลี่ยนแปลงไปอย่างไร? (เพิ่มขึ้น / ลดลง)
 - การเปลี่ยนแปลงนี้ จะส่งผลต่อความแม่นยำของโมเดลอย่างไร? .
.....
2. การปรับเกณฑ์ความมั่นใจ (Thresholding):
 - ไปที่ไฟล์ run_model.py บรรทัดที่เช็คเงื่อนไข if best_score > 0.6:
 - การทดลอง A: เปลี่ยนตัวเลขจาก 0.6 เป็น 0.9 แล้วนำหน้าตัวเองไปส่งกล่อง ผลลัพธ์ที่ได้คือ
.....
 - การทดลอง B: เปลี่ยนตัวเลขเป็น 0.2 แล้วให้คนแปลกหน้ามาส่งกล่อง ผลลัพธ์ที่ได้คือ
.....
 - จากการทดลอง สรุปได้ว่า หากเรานำระบบนี้ไปใช้กับประตูสแกนหน้าเข้าตึกที่มีความปลอดภัยสูง ค่า Threshold ควรจะ (สูง / ต่ำ) เพราะอะไร.....
.....